
Experiment 10

Optimal Power Flow using Steepest Descent Method in MATLAB and PSSE

Lab Report

Student Name HAMZA AHMAD

Registration Number 2022-EE-165

Section B

Power System Operation and Control Laboratory

Fall 2025

MATLAB code for steepest descent OPF?

```
clear; clc; close all;

%% SYSTEM DATA

BMva = 100;

% Columns: Bus | Vsp | delta | Pg | Qg | Pl | Ql | Qmin | Qmax | Type
busdata = [
    1   1.060   0   232.4  -16.9   0.0   0.0   0   0   1;
    2   1.045   0   40.0   43.56  21.7  12.7  -40  50  2;
    3   1.010   0    0.0    0.0  94.2  19.0  -25  40  3;
    4   1.000   0    0.0    0.0  47.8  -3.9   0   0  3;
    5   1.000   0    0.0    0.0   7.6   1.6   0   0  3;
```

```

        6   1.070   0       0.0   0.0   11.2   7.5   -6   24   2;
        7   1.000   0       0.0   0.0   0.0    0.0    0    0   3;
        8   1.090   0       0.0   0.0   0.0    0.0    0    0   2;
        9   1.000   0       0.0   0.0   29.5   16.6    0    0   3;
       10   1.000   0       0.0   0.0   9.0    5.8    0    0   3;
       11   1.000   0       0.0   0.0   3.5    1.8    0    0   3;
       12   1.000   0       0.0   0.0   6.1    1.6    0    0   3;
       13   1.000   0       0.0   0.0   13.5   5.8    0    0   3;
       14   1.000   0       0.0   0.0   14.9   5.0    0    0   3;

];

```

```

% line data

```

```

linedata = [
    1   2   0.01938   0.05917   0.0264   1.000;
    1   5   0.05403   0.22304   0.0246   1.000;
    2   3   0.04699   0.19797   0.0219   1.000;
    2   4   0.05811   0.17632   0.0170   1.000;
    2   5   0.05695   0.17388   0.0173   1.000;
    3   4   0.06701   0.17103   0.0064   1.000;
    4   5   0.01335   0.04211   0.0000   1.000;
    4   7   0.00000   0.20912   0.0000   0.978;
    4   9   0.00000   0.55618   0.0000   0.969;
    5   6   0.00000   0.25202   0.0000   1.000;
    6  11   0.09498   0.19890   0.0000   1.000;
    6  12   0.12291   0.25581   0.0000   1.000;
    6  13   0.06615   0.13027   0.0000   1.000;
    7   8   0.00000   0.17615   0.0000   1.000;
    7   9   0.00000   0.11001   0.0000   1.000;
    9  10   0.03181   0.08450   0.0000   1.000;
    9  14   0.12711   0.27038   0.0000   1.000;
   10  11   0.08205   0.19207   0.0000   1.000;
   12  13   0.22092   0.19988   0.0000   1.000;
   13  14   0.17093   0.34802   0.0000   1.000;

];

```

```

%% opf parametrers

a_cost = 120;      b_cost = 25;      c_cost = 0.01;

PG2_min = 10;

PG2_max = 100;

% (1-5)

constraint_line_from = 1;

constraint_line_to = 5;

P_line_max = 100;

alpha_init = 0.5;  alpha_min = 0.001;  alpha_decay = 0.95;

epsilon = 0.01;    max_iter_opf = 50;

penalty_factor = 1000;

%% initial load flow

fprintf('step 1: initial nr lf...\n');

[V_init, delta_init, P_init, Q_init, Ploss_init, Y, G, B] = ...
    run_load_flow(busdata, linedata, BMva);

P_line_init = calculate_line_flow(1, 5, V_init, delta_init, linedata,
BMva);

% calc initial cost

PG2_init = busdata(2, 4); % initial pg2 in mw

cost_init = calculate_cost(PG2_init, a_cost, b_cost, c_cost);

fprintf('\ninitial system state:\n');

fprintf('  PG2 = %.2f MW\n', PG2_init);

fprintf('  ine 1-5 Flow = %.2f MW (Limit: %.2f MW)\n', P_line_init,
P_line_max);

fprintf('  sys loss = %.2f MW\n', Ploss_init * BMva);

fprintf('  gene cost = $%.2f/hr\n', cost_init);

fprintf('  slack Gen (PG1) = %.2f MW\n\n', P_init(1) * BMva);

%% lossless ecomical dispatch

P_load_total = sum(busdata(:, 6)); % mw

% guess

PG2_initial = 60;

busdata(2, 4) = PG2_initial;

fprintf('starting PG2 (ED): %.2f MW\n\n', PG2_initial);

%% steepest descent

```

```

PG2_history = zeros(max_iter_opf, 1);
cost_history = zeros(max_iter_opf, 1);
gradient_history = zeros(max_iter_opf, 1);
line_flow_history = zeros(max_iter_opf, 1);
slack_gen_history = zeros(max_iter_opf, 1);

% curr val
PG2_current = PG2_initial;
alpha = alpha_init;
converged = false;
tic_opf = tic;
for iter = 1:max_iter_opf
    fprintf('Iteration %d:\n', iter);
    fprintf('  PG2 = %.4f MW\n', PG2_current);
    busdata(2, 4) = PG2_current;

    % run lf      <_> <_>
    [V, delta, P, Q, Ploss, ~, ~, ~] = run_load_flow(busdata, linedata,
BMva);

    % cal lf on 1-5
    P_line = calculate_line_flow(constraint_line_from, constraint_line_to,
...
                                V, delta, linedata, BMva);
    cost_current = calculate_cost(PG2_current, a_cost, b_cost, c_cost);
    grad_cost = b_cost + 2 * c_cost * PG2_current;

    % calculate sensitivity =dP_line/dPG2 using finite difference
    delta_PG = 0.1;
    busdata(2, 4) = PG2_current + delta_PG;
    [V_pert, delta_pert, ~, ~, ~, ~, ~, ~] = run_load_flow(busdata,
linedata, BMva);

    P_line_pert = calculate_line_flow(constraint_line_from,
constraint_line_to, ...
                                    V_pert, delta_pert, linedata, BMva);

    sensitivity = (P_line_pert - P_line) / delta_PG;

```

```

busdata(2, 4) = PG2_current;

% <oo> // !!!!
constraint_violation = max(0, P_line - P_line_max);

if constraint_violation > 0
    grad_penalty = penalty_factor * constraint_violation * sensitivity;
    grad_total = grad_cost + grad_penalty;
    fprintf(' uffff...constraint VIOLATED by %.2f MW\n',
constraint_violation);
else
    grad_total = grad_cost;
    fprintf(' constraint satisfied..okkk\n');
end

PG2_history(iter) = PG2_current;
cost_history(iter) = cost_current;
gradient_history(iter) = grad_total;
line_flow_history(iter) = P_line;
slack_gen_history(iter) = P(1) * BMva;

fprintf(' cost = $%.2f/hr\n', cost_current);
fprintf(' gradient = %.4f\n', grad_total);
fprintf('line 1-5 Flow = %.2f MW\n', P_line);
fprintf(' Slack Gen = %.2f MW\n', P(1) * BMva);

PG2_new = PG2_current - alpha * grad_total;
if PG2_new < PG2_min
    PG2_new = PG2_min;
    fprintf(' generator limit enforced min\n');
elseif PG2_new > PG2_max
    PG2_new = PG2_max;
    fprintf(' generator limit enforced max\n');

```

```

end

% check convergence

change = abs(PG2_new - PG2_current);

fprintf('  Change in PG2 = %.4f MW\n', change);

fprintf('  Step size alpha = %.4f\n', alpha);

% check bound with gradient pointing outward KKT

at_lower_bound = (PG2_current <= PG2_min + 0.01) && (grad_total > 0);
at_upper_bound = (PG2_current >= PG2_max - 0.01) && (grad_total < 0);

if (change < epsilon && abs(grad_total) < 1.0)
    converged = true;

    fprintf('\nCONV ACHIEVED hahahaha is!\n\n');

    % Store final values

    PG2_history(iter+1) = PG2_new;

    busdata(2, 4) = PG2_new;

    [V, delta, P, Q, Ploss, ~, ~, ~] = run_load_flow(busdata, linedata,
BMva);

    cost_history(iter+1) = calculate_cost(PG2_new, a_cost, b_cost,
c_cost);

    line_flow_history(iter+1) =
calculate_line_flow(constraint_line_from, ...

                    constraint_line_to, V, delta, linedata,
BMva);

    slack_gen_history(iter+1) = P(1) * BMva;

    gradient_history(iter+1) = grad_total;

    iter = iter + 1;

    break;

elseif (at_lower_bound || at_upper_bound) && change < epsilon
    converged = true;

    if at_lower_bound

        fprintf('\noptimal sol at min bound = KKT conditions
satisfied\n');

        fprintf('  gradient = %.4f > 0 indicates cost decreases with
lower PG2\n', grad_total);

```

```

else

    fprintf('\n ptimal sol at max boundd !\n');

    fprintf(' gradient = %.4f < 0 indicates cost decreases with
higher PG2\n', grad_total);

end

fprintf(' constrained optimum reached.\n\n');

% Store final values

PG2_history(iter+1) = PG2_new;

busdata(2, 4) = PG2_new;

[V, delta, P, Q, Ploss, ~, ~, ~] = run_load_flow(busdata, linedata,
BMva);

cost_history(iter+1) = calculate_cost(PG2_new, a_cost, b_cost,
c_cost);

line_flow_history(iter+1) =
calculate_line_flow(constraint_line_from, ...
                    constraint_line_to, V, delta, linedata,
BMva);

slack_gen_history(iter+1) = P(1) * BMva;

gradient_history(iter+1) = grad_total;

iter = iter + 1;

break;

end

% update for next iteration

PG2_current = PG2_new;

% Adapt step size

alpha = max(alpha * alpha_decay, alpha_min);

end

time_opf = toc(tic_opf);

PG2_history = PG2_history(1:iter);

cost_history = cost_history(1:iter);

gradient_history = gradient_history(1:iter);

line_flow_history = line_flow_history(1:iter);

slack_gen_history = slack_gen_history(1:iter);

```

```

%% final result

fprintf('  OPTIMIZATION RESULTS\n');

fprintf('=====\\n');

if converged

    fprintf('status: conv in %d iterations\\n', iter-1);
else

    fprintf('status: max itera reach (%d)\\n', iter);
end

fprintf('comp Time: %.4f seconds\\n\\n', time_opf);

fprintf('before optimization:\\n');

fprintf('  PG2 = %.2f MW\\n', PG2_init);

fprintf('  Cost = $%.2f/hr\\n', cost_init);

fprintf('  Line 1-5 Flow = %.2f MW\\n', P_line_init);

fprintf('\\n');

fprintf('after optimization:\\n');

fprintf('  PG2 = %.2f MW\\n', PG2_history(end));

fprintf('  cost = $%.2f/hr\\n', cost_history(end));

fprintf('  cost Reduction = $%.2f/hr\\n', cost_init - cost_history(end));

fprintf('  line 1-5 Flow = %.2f MW (Limit: %.2f MW)\\n', ...

        line_flow_history(end), P_line_max);

fprintf('  slack gen (PG1) = %.2f MW\\n', slack_gen_history(end));

fprintf('  system Loss = %.2f MW\\n', Ploss * BMva);

if line_flow_history(end) <= P_line_max

    fprintf('  line constraint satisfy\\n');
else

    fprintf('  line constraint not satisfy\\n');
end

%% final power flow

fprintf('FINAL BUS VOLTAGE AND ANGLE:\\n');

fprintf('-----\\n');

fprintf(' Bus | Voltage (pu) | Angle (deg)  \\n');

fprintf('-----\\n');

for i = 1:length(V)

```



```

        fprintf(' %2d | %7.4f | %9.4f\n', i, V(i), rad2deg(delta(i)));
end

fprintf('-----\n\n');

%% ===== COMPARISON TABLE =====

fprintf('detail table compariosn:\n');

fprintf('=====\n');

fprintf('parameter | initial | ptimized | change\n');

fprintf('=====\n');

fprintf('PG2 (MW) | %8.2f | %8.2f | %7.2f\n', ...
        PG2_init, PG2_history(end), PG2_history(end)-PG2_init);

fprintf('Cost ($/hr) | %8.2f | %8.2f | %7.2f\n', ...
        cost_init, cost_history(end), cost_history(end)-cost_init);

fprintf('Line 1-5 Flow (MW) | %8.2f | %8.2f | %7.2f\n', ...
        P_line_init, line_flow_history(end),
        line_flow_history(end)-P_line_init);

fprintf('Slack Gen PG1 (MW) | %8.2f | %8.2f | %7.2f\n', ...
        P_init(1)*BMva, slack_gen_history(end),
        slack_gen_history(end)-P_init(1)*BMva);

fprintf('System Loss (MW) | %8.2f | %8.2f | %7.2f\n', ...
        Ploss_init*BMva, Ploss*BMva, Ploss*BMva-Ploss_init*BMva);

fprintf('-----\n\n');

fprintf('opf completed\n\n');

%% functions

function [V, delta, P, Q, Ploss, Y, G, B] = run_load_flow(busdata,
linedata, BMva)

nbus = max(max(linedata(:, 1:2)));

Y = zeros(nbus, nbus);

fb = linedata(:, 1);
tb = linedata(:, 2);

for k = 1:size(linedata, 1)

    i = fb(k);

    j = tb(k);

    R = linedata(k, 3);

```

```

X = linedata(k, 4);
Bc = linedata(k, 5);
tap = linedata(k, 6);

if tap == 0
    tap = 1;
end

Z = R + 1i * X;
y = 1 / Z;
ysh = 1i * Bc;

Y(i, j) = Y(i, j) - y / tap;
Y(j, i) = Y(i, j);
Y(i, i) = Y(i, i) + y / (tap^2) + ysh;
Y(j, j) = Y(j, j) + y + ysh;
end

G = real(Y);
B = imag(Y);
type = busdata(:, 10);
V = busdata(:, 2);
delta = deg2rad(busdata(:, 3));
Pg = busdata(:, 4) / BMva;
Qg = busdata(:, 5) / BMva;
Pl = busdata(:, 6) / BMva;
Ql = busdata(:, 7) / BMva;

Psp = Pg - Pl;
Qsp = Qg - Ql;

pq = find(type == 3);
npq = length(pq);

```

```

tol = 1e-6;
Iter = 0;
Tol = 1;
while Tol > tol && Iter < 100
    Iter = Iter + 1;

    P = zeros(nbus, 1);
    Q = zeros(nbus, 1);

    for i = 1:nbus
        for k = 1:nbus
            P(i) = P(i) + V(i) * V(k) * (G(i,k) * cos(delta(i) -
delta(k)) + ...
                                B(i,k) * sin(delta(i) -
delta(k)));
            Q(i) = Q(i) + V(i) * V(k) * (G(i,k) * sin(delta(i) -
delta(k)) - ...
                                B(i,k) * cos(delta(i) -
delta(k)));
        end
    end

    dP = Psp - P;
    dQ = Qsp - Q;
    M = [dP(2:end); dQ(pq)];
    Tol = max(abs(M));

    if Tol < tol
        break;
    end

    J1 = zeros(nbus - 1);
    for i = 1:nbus - 1
        m = i + 1;
        for k = 1:nbus - 1

```

```

n = k + 1;

if n == m

    for n2 = 1:nbus

        J1(i, k) = J1(i, k) + V(m) * V(n2) * ...

            (-G(m,n2) * sin(delta(m) - delta(n2)) +

...

            B(m,n2) * cos(delta(m) - delta(n2)));

    end

    J1(i, k) = J1(i, k) - V(m)^2 * B(m, m);

else

    J1(i, k) = V(m) * V(n) * ...

        (G(m,n) * sin(delta(m) - delta(n)) - ...

        B(m,n) * cos(delta(m) - delta(n)));

    end

end

end

J2 = zeros(nbus - 1, npq);

for i = 1:nbus - 1

    m = i + 1;

    for k = 1:npq

        n = pq(k);

        if n == m

            for n2 = 1:nbus

                J2(i, k) = J2(i, k) + V(n2) * ...

                    (G(m,n2) * cos(delta(m) - delta(n2)) + ...

                    B(m,n2) * sin(delta(m) - delta(n2)));

            end

            J2(i, k) = J2(i, k) + V(m) * G(m, m);

        else

            J2(i, k) = V(m) * ...

                (G(m,n) * cos(delta(m) - delta(n)) + ...

                B(m,n) * sin(delta(m) - delta(n)));

        end
    end
end

```

```

        end
    end
end

J3 = zeros(npq, nbus - 1);
for i = 1:npq
    m = pq(i);
    for k = 1:nbus - 1
        n = k + 1;
        if n == m
            for n2 = 1:nbus
                J3(i, k) = J3(i, k) + V(m) * V(n2) * ...
                    (G(m,n2) * cos(delta(m) - delta(n2)) + ...
                    B(m,n2) * sin(delta(m) - delta(n2)));
            end
            J3(i, k) = J3(i, k) - V(m)^2 * G(m, m);
        else
            J3(i, k) = V(m) * V(n) * ...
                (-G(m,n) * cos(delta(m) - delta(n)) - ...
                B(m,n) * sin(delta(m) - delta(n)));
        end
    end
end
end

```

```

J4 = zeros(npq);
for i = 1:npq
    m = pq(i);
    for k = 1:npq
        n = pq(k);
        if n == m
            for n2 = 1:nbus
                J4(i, k) = J4(i, k) + V(n2) * ...
                    (G(m,n2) * sin(delta(m) - delta(n2)) - ...

```

```

        B(m,n2) * cos(delta(m) - delta(n2)));

    end

    J4(i, k) = J4(i, k) - V(m) * B(m, m);

else
    J4(i, k) = V(m) * ...
        (G(m,n) * sin(delta(m) - delta(n)) - ...
        B(m,n) * cos(delta(m) - delta(n)));

end

end

end

end

J = [J1 J2; J3 J4];
X = J \ M;

dTheta = X(1:nbus - 1);
dV = X(nbus:end);
delta(2:end) = delta(2:end) + dTheta;
V(pq) = V(pq) + dV;

end

Ploss = 0;

for k = 1:size(linedata, 1)
    i = fb(k);
    j = tb(k);
    R = linedata(k, 3);
    X = linedata(k, 4);
    tap = linedata(k, 6);

    if tap == 0
        tap = 1;
    end

    Z = R + 1i * X;
    y = 1 / Z;

```

```

    Vi = V(i) * exp(1i * delta(i));
    Vj = V(j) * exp(1i * delta(j));

    Iij = (Vi - Vj / tap) * y;
    Iji = (Vj - Vi * tap) * y;

    Sij = Vi * conj(Iij);
    Sji = Vj * conj(Iji);

    Ploss = Ploss + real(Sij + Sji);
end

end

function P_line = calculate_line_flow(from_bus, to_bus, V, delta, linedata,
BMva)

    % cal pf on line specf

    % Find the line

    line_idx = find((linedata(:,1) == from_bus & linedata(:,2) == to_bus) |
...
                    (linedata(:,1) == to_bus & linedata(:,2) == from_bus));

    if isempty(line_idx)
        error('Line %d-%d not found', from_bus, to_bus);
    end

    line_idx = line_idx(1); % Take first match

    i = linedata(line_idx, 1);
    j = linedata(line_idx, 2);
    R = linedata(line_idx, 3);
    X = linedata(line_idx, 4);
    tap = linedata(line_idx, 6);

```

```

if tap == 0
    tap = 1;
end

Z = R + 1i * X;
y = 1 / Z;

Vi = V(i) * exp(1i * delta(i));
Vj = V(j) * exp(1i * delta(j));

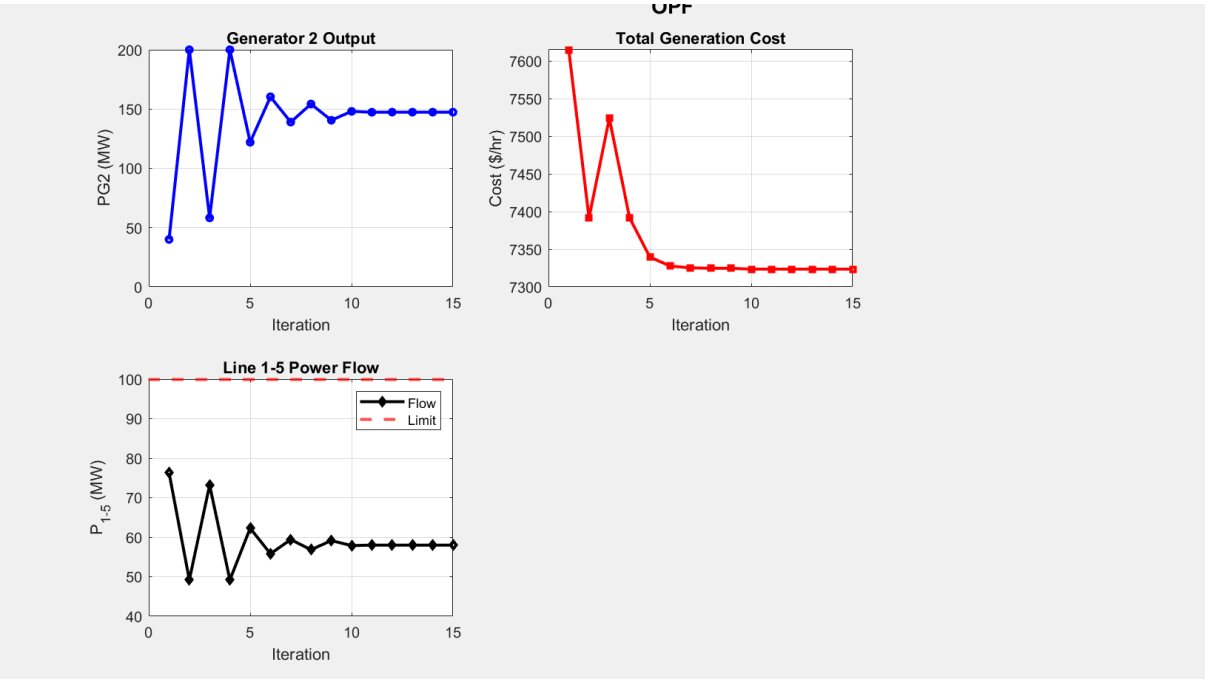
Iij = (Vi - Vj / tap) * y;
Sij = Vi * conj(Iij);

P_line = real(Sij) * BMva; % con to MW
end

function cost = calculate_cost(PG, a, b, c)
    cost = a + b * PG + c * PG^2;
end

```

Convergence plot of PG2?



Final cost and constraint verification ?

step 1: initial nr lf...

initial system state:

PG2 = 40.00 MW

ine 1-5 Flow = 76.32 MW (Limit: 100.00 MW)

sys loss = 15.88 MW

gene cost = \$1136.00/hr

slack Gen (PG1) = 232.69 MW

starting PG2 (ED): 60.00 MW

Iteration 1:

PG2 = 60.0000 MW

constraint satisfied..okkk

cost = \$1656.00/hr

gradient = 26.2000

line 1-5 Flow = 72.87 MW

Slack Gen = 211.67 MW

fx

Change in PG2 = 13.1000 MW

Command Window

```
gradient = 26.2000  
line 1-5 Flow = 72.87 MW  
Slack Gen = 211.67 MW  
Change in PG2 = 13.1000 MW  
Step size alpha = 0.5000
```

Iteration 2:

```
PG2 = 46.9000 MW  
constraint satisfied..okkk  
cost = $1314.50/hr  
gradient = 25.9380
```

```
line 1-5 Flow = 75.13 MW  
Slack Gen = 225.42 MW  
Change in PG2 = 12.3205 MW  
Step size alpha = 0.4750
```

Iteration 3:

```
PG2 = 34.5795 MW  
constraint satisfied..okkk  
cost = $996.44/hr  
gradient = 25.6916
```

fx



Command Window

```
PG2 = 34.5795 MW
constraint satisfied..okkk
cost = $996.44/hr
gradient = 25.6916
line 1-5 Flow = 77.26 MW
Slack Gen = 238.42 MW
Change in PG2 = 11.5933 MW
Step size alpha = 0.4512
Iteration 4:
PG2 = 22.9861 MW
constraint satisfied..okkk
cost = $699.94/hr
gradient = 25.4597
line 1-5 Flow = 79.28 MW
Slack Gen = 250.69 MW
Change in PG2 = 10.9143 MW
Step size alpha = 0.4287
Iteration 5:
fx PG2 = 12.0719 MW
```

Command Window

OPTIMIZATION RESULTS

=====

status: conv in 6 iterations

comp Time: 0.0227 seconds

before optimization:

PG2 = 40.00 MW

Cost = \$1136.00/hr

Line 1-5 Flow = 76.32 MW

after optimization:

PG2 = 10.00 MW

cost = \$371.00/hr

cost Reduction = \$765.00/hr

line 1-5 Flow = 81.54 MW (Limit: 100.00 MW)

slack gen (PG1) = 264.49 MW

system Loss = 17.67 MW

line constraint satisfy

FINAL BUS VOLTAGE AND ANGLE:

Bus	Voltage (pu)	Angle (deg)

1	1.0600	0.0000
2	1.0450	-5.8432
3	0.9855	-13.2076
4	1.0162	-11.0020
5	1.0264	-9.5377
6	1.0700	-15.4141
7	1.0514	-14.0593
8	1.0900	-14.0593
9	1.0349	-15.6735
10	1.0336	-15.9102
11	1.0480	-15.7804
12	1.0536	-16.2533
13	1.0473	-16.2830
14	1.0221	-16.9630

fx

detail table compariosn:

=====

parameter	initial	ptimized	change
=====			
PG2 (MW)	40.00	10.00	-30.00
Cost (\$/hr)	1136.00	371.00	-765.00
Line 1-5 Flow (MW)	76.32	81.54	+5.22
Slack Gen PG1 (MW)	232.69	264.49	+31.80
System Loss (MW)	15.88	17.67	+1.79

opf completed